The Benchmark Chooses the Winner: Operating Points, Fine-Tuning, and the Fragility of Safety-Guard Rankings

A Fair, Reproducible Evaluation of Small LLM Guards

Reza Rahimi
reza.rahimi@jazzx.ai
JazzX AI
Los Altos, CA, USA

ABSTRACT

Safety guards are typically compared by a single F1 computed at each model's own decision threshold. We show this turns the ranking into an artifact of the operating point: the reported winner changes with the threshold, the metric, and the benchmark. Treating that as the central measurement problem, we study a small, laptop-trained guard — SmolLM3-3B fine-tuned with LoRA into a one-forward-pass {safe, unsafe} classifier — under an operating-point-fair protocol: threshold-free AUPRC/AUROC, a matched-FPR operating point, and paired-bootstrap 95% confidence intervals (CIs) with McNemar tests. Building this protocol also surfaced and fixed real evaluation bugs in our own first pipeline (calibration leakage, batch-amortized latency, an asymmetric per-model threshold advantage, and a baseline error-handling bias).

Three results make the thesis concrete. First, native-threshold F1 is rank-inverting: two open guards swap order between their native F1 and threshold-free AUPRC. Second, a base-vs-tuned decomposition shows parameter-efficient fine-tuning specializes in-distribution while *degrading* out-of-distribution ranking — on four novel benchmarks the *untuned* base outranks the tuned guard (aggregate AUPRC 0.886 vs 0.781, non-overlapping CIs) at equal peak F1, so the guard's out-of-distribution lead is inherited base competence rather than a product of tuning. Third, an *exploratory*, seed-scale mortgage case study illustrates that the winner is benchmark-dependent: a saturated in-domain benchmark ranks every system near the ceiling, whereas a hardened, trap-typed rebuild de-saturates the field — small guards over-block benign-but-risky prompts far more than a frontier model, and the in-domain fine-tune that leads *here* is the reverse of the general-OOD result where the untuned base leads. Under the fair protocol the guard still beats Llama-Guard-3-1B decisively (non-overlapping CIs, including out-of-distribution) and ties gpt-5.4-mini on F1 at lower batch=1 latency: an efficiency result, not an accuracy one.

1 INTRODUCTION

Large language models are now deployed as chat assistants and autonomous agents, where a single unsafe completion, a successful jailbreak, or an over-eager refusal all carry real cost. The dominant mitigation is a *safety guard*: a classifier that screens content — most commonly user prompts, and in some systems model responses — as `safe` or `unsafe` before it reaches the model or the outside world. We are explicit up front about scope: our guard and *every* evaluation in this paper classify **input prompts**. Screening of model *responses* and of *agent* tool-call arguments or tool outputs — the surfaces that motivate the "agent-bouncer" name — is *not* evaluated here and is left to future work (§6.1). In practice, teams face an unattractive choice. Purpose-built open guards (Llama Guard 1/2/3 and Prompt Guard, ShieldGemma, Aegis, Granite Guardian, WildGuard) are convenient but are often reported on their own favorable operating points; proprietary frontier classifiers are strong but add per-request cost, latency, and a data-sharing dependency. Meanwhile, published guard comparisons frequently tune a threshold for the proposed system while holding baselines at fixed points, report latency as batch-amortized throughput, and decontaminate with exact string matching — choices that quietly inflate the reported advantage.

This paper is a **measurement and controlled-study** contribution, not a state-of-the-art chase. We fine-tune SmolLM3-3B into a binary guard that emits a single-token verdict — a softmax over the `{safe, unsafe}` token logits at the last position, i.e. one forward pass — with temperature and decision threshold calibrated on an *in-distribution* dev split only (in-house held-out and novel benchmarks contribute zero rows to calibration). Training is parameter-efficient LoRA (r=32, all seven projections; 60.5M trainable params, 1.93% of 3.14B) run in ~71 minutes / 300 steps on a single consumer Apple M4 Max laptop (36 GB unified memory, MPS backend, no CUDA). Everything — training, calibration, and evaluation — fits on a laptop, so the protocol is cheap to reproduce and audit.

Our emphasis is on *how* we measure, and we are careful with the word "held-out," which we split into two distinct pools throughout the paper:

- **In-house held-out** — two of our six in-house benchmarks (JailbreakBench, XSTest) are held out from training and calibration but drawn from the same overall pool.
- **Novel held-out (OOD)** — four benchmarks from source families the guard never saw during training (WildGuardTest, WildJailbreak, OR-Bench-Hard, HarmBench).

We evaluate on 6 in-house benchmarks (2,018 balanced, leakage-filtered test rows across guardrail, red-team, and over-refusal axes) and on the 4 novel held-out benchmarks (2,020 balanced rows across three sets, plus 200 all-harmful HarmBench rows). We compare against gpt-5.4-mini, Llama-Guard-3-1B, ShieldGemma-2b, and a keyword matcher, using paired-bootstrap 95% CIs, McNemar tests, threshold-free AUPRC/AUROC, and a matched-FPR@0.10 operating point, with latency measured at true batch=1. Building this pipeline surfaced real bugs in our own first evaluation — a calibration leak, batch-amortized latency, an asymmetric threshold advantage for our own model, and an

API-error default that biased the GPT baseline's FPR — which we document and fix.

The headline comparisons are collected once in Table 1 and reported in full, with CIs, in §5; we do not restate them in prose. The one-line reading: under the fair protocol the guard leads the open guards on threshold-free AUPRC — decisively over Llama-Guard-3-1B (non-overlapping CIs), including on the four novel benchmarks — and ties gpt-5.4-mini on F1 at lower batch=1 latency. But the load-bearing finding is not the guard's standing; it is that the standing is contingent on the operating point, and that the tuning credited for it does not survive out of distribution: the *untuned* base outranks the tuned guard on the novel benchmarks (§5.5).

We are explicit about limits. Matched-FPR@0.10 makes every system conservative (guard recall 0.431 there), so AUPRC is our fair primary summary; at the *deployed* operating point (T=2.10, $\tau = 0.59$) the guard's overall FPR is 0.306, i.e. it over-blocks benign prompts, which we treat as a real deployment concern. ShieldGemma is run somewhat outside its designed content-policy regime and its novel-benchmark numbers are still pending; a single global threshold is structurally in-distribution-optimized. Positioned against the two closest works, we complement CPU-class efficient-guard studies ("Do You Really Need a GPU to Guard Your LLM?") by adding rigorous operating-point-fair comparison and a base-vs-tuned decomposition on a 3B generative guard, and we serve as the constructive counterpart to "Why LLM Safety Guardrails Collapse After Fine-tuning" (ICML 2025): where they show tuning can destroy alignment, we show that even when tuning *works*, its measurable gains are memorized in-distribution and do not transfer to the in-house held-out sets we can test.

1.1 Contributions

- **Operating-point-fair evaluation.** We show guard rankings depend heavily on the decision threshold, so naive per-model-threshold comparisons are misleading; we instead report matched-FPR@0.10, threshold-free AUPRC/AUROC, and paired-bootstrap CIs with McNemar tests.
- **A base-vs-tuned decomposition.** An instruction-tuned 3B base already carries most of the safety-classification competence measurable on our in-house benchmarks by F1 (its zero-shot F1 beats both open guards'); parameter-efficient tuning specializes in-distribution and does *not* meaningfully improve the two in-house held-out benchmarks we can measure (macro −0.005), and slightly regresses over-refusal.
- **A domain stress test of the same effect (exploratory).** On a mortgage fair-lending task we show a benchmark can be *saturated* — a zero-shot 3B base is already near-ceiling, so every system looks excellent and none can be ranked — and we rebuild it as a hardened, trap-typed set that de-saturates the field: small guards over-block benign-adjacent prompts far more than a frontier model, and which fine-tune "wins" flips between this in-domain set and the general novel sets. We flag this as a seed-scale, single-annotator case study rather than a headline result (Section 5.6).
- **A small, local, laptop-trained guard.** On threshold-free AUPRC it surpasses purpose-built open guards — Llama-Guard-3-1B decisively (non-overlapping CIs), *including* on the novel held-out benchmarks, and ShieldGemma-2b on the in-house pooled / in-dist / in-house-held-out sets (ShieldGemma's novel runs are pending) — and it matches frontier gpt-5.4-mini on F1 at ~4× lower latency.
- **A rigorous, reproducible, laptop-scale protocol.** Paired CIs, McNemar, matched-FPR, and family-level decontamination — a pipeline that exposed real evaluation bugs in our own first attempt — released as open code (the evaluation scripts plus a single orchestrating reproduction notebook).

2 RELATED WORK

LLM safety guards and content moderation. A growing family of purpose-built classifiers screens prompts and responses for LLM and agent pipelines. Meta's Llama Guard line (1/2/3) [7, 15, 24] and Prompt Guard [23], Google's ShieldGemma [30], NVIDIA's Aegis [8], IBM's Granite Guardian [25], and AI2's WildGuard [11] all frame moderation as a supervised classification task over a safety taxonomy, typically emitting a verdict token or category label. Recent work pushes on the *reasoning* axis (R^2-Guard's knowledge-enhanced logical-reasoning inference [18] at ICLR 2025, DuoGuard's two-player RL/multilingual training [5], and other reasoning-guards). Our guard is deliberately minimal by comparison: a single-token verdict read from the {safe, unsafe} logits in one forward pass, LoRA-tuned on a consumer laptop. We compare directly against two open guards under a fair protocol; on threshold-free AUPRC our guard leads both open guards on the in-house pool (full comparison with CIs in Table 5; per-pooling breakdown in Table 12).

Benchmarks. We evaluate on established suites spanning guardrail, red-team, and over-refusal axes: BeaverTails [16], ToxicChat [19], JailbreakBench [3], XSTest [27], OR-Bench [4], HarmBench [22], and prompt-injection/jailbreak-classification sets, plus OpenAI's moderation evaluation set [21]. To probe generalization rather than memorization, we additionally reserve four novel benchmarks the guard never trained on — WildGuardTest [11], WildJailbreak [17], OR-Bench-Hard [4], and HarmBench [22] — held out at the source-family level, and report per-benchmark and aggregate results with paired-bootstrap 95% CIs and McNemar tests. On these four novel held-out benchmarks the guard's aggregate AUPRC is 0.781 [0.751, 0.811] versus Llama-Guard-3-1B 0.701 [0.673, 0.733], the only cross-system AUPRC comparison in which both sides carry CIs and they do not overlap.

Measurement rigor. A recurring hazard in guard evaluation is that rankings are sensitive to the decision threshold, and that a model tuned on its own dev split is compared against baselines evaluated at fixed operating points. We treat this as a first-class methodological concern rather than an implementation detail: we report threshold-free AUPRC/AUROC, matched-FPR@0.10 operating points, true batch=1 latency (p50 124 ms / p90 188 ms), paired CIs, and family-level holdout. This positions our work as a *measurement and controlled study* rather than a leaderboard entry.

Table 1: Headline results (in-house pooled unless noted). Only the guard's pooled/held-out AUPRC, the guard-vs-Llama novel aggregate, the base-vs-tuned delta, and the GPT ΔF1 carry CIs; baseline AUPRCs are point estimates.

Metric (in-house pooled unless noted)	Guard	Llama-Guard-3-1B	ShieldGemma-2b	gpt-5.4-mini
AUPRC	0.844 [0.825, 0.866]	0.639	0.712	—
AUPRC, 4 novel held-out (aggregate)	0.781 [0.751, 0.811]	0.701 [0.673, 0.733]	pending	—
Matched-FPR@0.10 F1	0.581	0.360	0.464	—
F1 (paired vs guard)	tie (ΔF1 +0.010, $p = 0.20$)	—	—	native point 0.784
Latency, batch=1 p50	124 ms	not measured	not measured	~512 ms (remote API)

2.1 Differentiation from the two closest works

"Do You Really Need a GPU to Guard Your LLM?" [20] (CPU-class classifiers / efficient pipelines). That line argues small, cheap classifiers can serve as practical guards. We share the efficiency motivation — our guard is laptop-trained (~71 min / 300 steps, bf16, Apple M4 Max, MPS, no CUDA) and runs locally at ~4× lower latency than gpt-5.4-mini (124 ms vs ~512 ms per request; see the latency caveat in Section 5.3). Our distinct contribution is not a smaller classifier but a *rigorous fair comparison* on a 3B generative guard: operating-point-fair evaluation (matched-FPR + threshold-free AUPRC + paired CIs) and a base-vs-tuned decomposition that prior efficient-guard work does not report.

"Why LLM Safety Guardrails Collapse After Fine-tuning" [13] (ICML 2025 DIG-BUGS). That work shows fine-tuning can *destroy* a model's alignment. We are the constructive complement. Even when fine-tuning "works," our base-vs-tuned ablation (identical paired rows; base = zero-shot SmolLM3-3B with the same logprob head) shows the measurable gains are almost entirely in-distribution and do not improve the two in-house held-out benchmarks we can test (per-benchmark decomposition in Table 9). The zero-shot base (F1 0.713) exceeds zero-shot Llama-Guard-3-1B (0.673) and ShieldGemma-2b (0.424) *by F1*, suggesting an instruction-tuned 3B base already carries much of the safety-classification competence; under threshold-free AUPRC, however, the base's in-house score (0.696) sits *between* the two open guards (Llama-Guard-3-1B 0.639, ShieldGemma-2b 0.712), so this in-house ranking holds by F1 only and not threshold-free (Section 5.5). Where the ICML work demonstrates that tuning can break alignment, we show that tuning which appears successful still only measurably specializes in-distribution, and we decompose how much competence is base versus tuned. We have since run the base on the four novel benchmarks (Section 5.5): the base *outranks* the tuned guard out-of-distribution (aggregate AUPRC 0.886 vs 0.781, disjoint CIs) at tied best-threshold F1, so the guard's novel-set lead reflects inherited base competence, and tuning degrades OOD score ranking rather than improving generalization.

2.2 What is novel here

Threshold-free AUPRC is itself an established guard metric [15, 18, 21, 25, 30], so we do **not** claim it as our innovation. What prior work lacks — and what we contribute — is the *combination*: matched-FPR across systems (nearest prior art, Granite Guardian's fixed-FPR points [25]), AUPRC *and* AUROC, paired-bootstrap CIs and McNemar tests on a shared calibration set [9], plus the finding that native-threshold F1 is not merely imprecise but rank-*inverting* (Section 5.7). Our two genuinely new findings are this operating-point-fairness result and the base-vs-tuned decomposition [13] (Section 5.5); the laptop-trained guard and the reproducible protocol are the engineering contributions listed above. We scope claims to detection-based content classification; adversarial evasion remains open [10].

3 METHODOLOGY

Figure 1 is the whole method at a glance — how the benchmark pool is split, the fine-tuning and calibration *technique*, and the frozen, operating-point-fair evaluation. It carries no result numbers (those live in the tables it points to); the rest of this section details each stage, and the hardware, software, and reproducibility setup is collected in Section 4.

3.1 Guard formulation: a single-token logprob head

The guard treats safety classification as a single next-token prediction. A prompt to be screened is wrapped in a fixed instruction template and the model is asked to emit a one-word verdict, `safe` or `unsafe`. Rather than sampling text, we read the logits at the final position and restrict attention to the two verdict tokens. Let z_{safe} and z_{unsafe} be those logits; the guard score is a temperature-scaled two-way softmax

$$P(\text{unsafe}) = \frac{\exp(z_{\text{unsafe}}/T)}{\exp(z_{\text{safe}}/T) + \exp(z_{\text{unsafe}}/T)}. \tag{1}$$

A single forward pass yields the score (Figure 2) — no autoregressive decoding, no multi-token or chain-of-thought generation. This keeps inference cheap and makes the guard a proper threshold-free scorer, so it can be summarized by AUPRC/AUROC in addition to a hard verdict. A prompt is flagged `unsafe` when $P(\text{unsafe}) \geq \tau$ for a fixed decision threshold τ.

3.2 Model selection: small language models

We use **SmolLM3-3B** [2] as the primary guard base — small enough to fine-tune and serve on a single consumer laptop, instruction-tuned (so it already carries much of the safety-classification competence, Section 5.5), and it tokenizes `safe`/`unsafe` as single tokens, which the one-forward-pass logprob head requires. The recipe is deliberately base-agnostic; to check that our conclusions are not an artifact of one model family, we apply the *identical* LoRA recipe and evaluation protocol to three further small instruction-tuned bases that

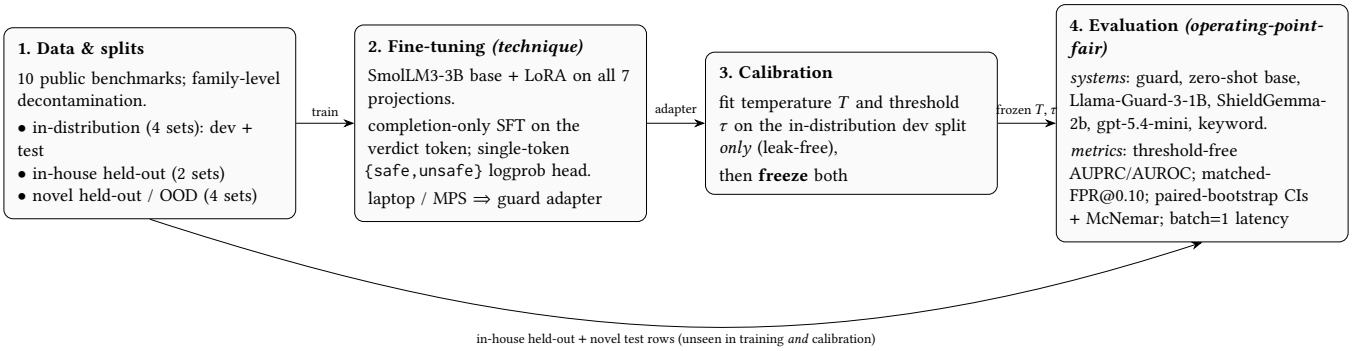

Figure 1: End-to-end setup. Only in-distribution *train* rows tune the LoRA guard and only in-distribution *dev* rows calibrate (T, τ), which are then frozen; every system is scored on the held-out and novel test rows under one operating-point-fair protocol. The zero-shot base is the same network with the LoRA adapter removed. Hyperparameters: Table 3; calibrated values: Table 2; results: §5.

Figure 2: The single-token logprob head. A prompt is wrapped in a fixed instruction template and passed once through the LoRA-adapted SmolLM3-3B; the last-position logits over {safe, unsafe} are turned into $P(\text{unsafe})$ by a temperature-scaled two-way softmax, and the prompt is flagged when $P(\text{unsafe}) \geq \tau$.

Table 2: Post-training calibration parameters, fit on the in-distribution dev split only.

Parameter	Value
Temperature T	2.10
Decision threshold τ	0.59

also expose single-token verdicts — Qwen2.5-1.5B-Instruct, DeepSeek-R1-Distill-Qwen-1.5B, and SmolLM2-1.7B-Instruct.

3.3 Calibration

The temperature T (temperature scaling [9]) and threshold τ are the only post-training free parameters, and both are fit exclusively on an **in-distribution dev split** drawn from the four in-distribution benchmarks (BeaverTails, ToxicChat, Prompt Injections, Jailbreak Classification). The two in-house held-out benchmarks (JailbreakBench, XSTest) and the four novel benchmarks contribute zero rows to calibration; this is a deliberate correction of an earlier pipeline in which held-out dev rows had leaked into temperature/threshold fitting. The clean, leak-free calibration yields the values in Table 2.

Because τ is tuned on in-distribution dev data, a single global threshold is structurally in-distribution-optimized; we treat this as a limitation and, for cross-model comparison, additionally report matched-FPR and threshold-free operating points rather than relying on the tuned point alone.

Table 3: LoRA fine-tuning recipe for the SmolLM3-3B guard.

Setting	Value
Base model	SmolLM3-3B (instruction-tuned)
Adapter	LoRA, rank $r = 32$, $\alpha = 64$, dropout 0.05
Target modules	q, k, v, o, gate, up, down
Loss	Completion-only, on the verdict token
Trainable params	60.5M (1.93% of 3.14B)
Precision	bf16
Optimizer steps	300
Wall-clock	~71 min
Seed	42

3.4 LoRA training recipe

We fine-tune SmolLM3-3B [2] with LoRA adapters [14] and a completion-only objective: the loss is computed on the verdict token only, so the model is trained to place probability mass on `safe`/`unsafe` at the decision position rather than to reproduce the prompt. Adapters are applied to all seven projection matrices (q, k, v, o, gate, up, down). Table 3 summarizes the recipe.

Training ran entirely on a consumer Apple M4 Max laptop (36 GB unified memory, MPS backend, no CUDA), demonstrating that the recipe is reproducible at laptop scale. Because ToxicChat is non-commercial, it is used for evaluation only and is excluded from any released model's training data.

3.5 Task and label mapping

The guard is a binary classifier that emits a single-token verdict, `safe` or `unsafe`. We read the logits at the last position, take a softmax over the {safe, unsafe} token pair, and treat the resulting $P(\text{unsafe})$ as the decision score. A prompt is labeled `unsafe` if it should be blocked (harmful content, successful jailbreak/injection, policy-violating request) and `safe` otherwise; the over-refusal axis contributes benign-but-superficially-risky prompts whose gold label is `safe`. All evaluations reduce to this one score per row, obtained in a single forward pass.

3.6 Benchmarks and terminology

We use three explicitly distinguished pools, and we reserve the word "held-out" for two of them:

- **In-distribution (in-dist):** BeaverTails, ToxicChat, Prompt Injections, Jailbreak Classification. Contribute dev rows to calibration plus balanced test rows.
- **In-house held-out:** JailbreakBench, XSTest. Same in-house pool, but test-only — zero dev/calibration rows.
- **Novel held-out (OOD):** WildGuardTest, WildJailbreak, OR-Bench-Hard, HarmBench. Held out at the source-family level; the guard never trained on them.

All splits are class-balanced (matched-n). HarmBench is an all-harmful stress set (no negatives), so it is scored by mean $P(\text{unsafe})$ rather than F1/AUPRC and is **excluded** from any aggregate AUPRC.

Selection. We pick established, publicly hosted sets that jointly cover the three safety axes — *guardrail* (harmful content), *red-team* (jailbreaks / prompt injection), and *over-refusal* (benign-but-superficially-risky) — and we represent each axis in *both* the in-house and the novel pool, so no axis is judged on a single source. XSTest is the over-refusal probe (its gold label is `safe`); HarmBench is an all-harmful stress set (no negatives), so it is scored by mean $P(\text{unsafe})$ and excluded from any aggregate AUPRC.

Train/dev/test separation (per benchmark). Only the four *in-distribution* sets contribute training and calibration data: each is split into a train portion (LoRA fine-tuning only), a dev portion (fitting T and τ only), and a class-balanced test portion; the **1,102** in-distribution dev rows come exclusively from these four (JailbreakBench and XSTest contribute *zero* dev/train rows). The two *in-house-held-out* sets and the four *novel* sets are **test-only**: the guard never sees them in training or calibration. Novel sets are additionally held out at the *source-family* level (the whole dataset family is excluded from training) with near-duplicate filtering, so held-out evaluation measures transfer rather than memorization. Every test split is class-balanced by matched-n sampling except XSTest (over-refusal, majority `safe`) and HarmBench (all-unsafe). Figure 3 shows the split-and-evaluation flow.

3.7 Baselines

We compare against four systems, each run in its own native regime:

- **gpt-5.4-mini** — zero-shot classifier prompt at a fixed native decision point (no tunable threshold). On API errors we abstain rather than defaulting to a label (a corrected bug; see Section 3.10).
- **Llama-Guard-3-1B** [24] — the standard (non-INT4) checkpoint, run with its native chat template, reading the first-token verdict. (The INT4-quantized variant of this 1B model is described separately in [7]; we do not use it.)
- **ShieldGemma-2b** — its guideline prompt, scored by $P(\text{Yes})$. ShieldGemma is a content-policy / general-guideline guard and is used somewhat outside its designed regime here; it is not an injection detector.
- **Keyword matcher** — a simple substring/keyword baseline.

For a matched comparison we also evaluate an untuned **base** SmolLM3-3B: zero-shot, with the identical single-token logprob head and no training, on the identical 2,018 test rows as the tuned guard. To keep the comparison fair, the base receives its *own* clean calibration — a base-specific temperature and threshold (T=7.00, τ=0.51), fit on the identical in-distribution dev split with the identical NLL-plus-F1 protocol used for the tuned guard — so each model is read at its own best in-distribution operating point rather than a shared or borrowed threshold. To avoid resting the base-vs-tuned claim on an operating point, we additionally report the base's threshold-free in-house AUPRC (Section 5.5).

3.8 Metrics and which comparisons carry CIs

We report F1 and false-positive rate (FPR) at the operating threshold, and the threshold-free summaries AUPRC and AUROC. Because only our guard receives a dev-tuned threshold while baselines are fixed operating points, we additionally compare all tunable systems at **matched-FPR@0.10** (threshold chosen so FPR = 0.10 on the in-distribution dev split) and rely on AUPRC as the fair threshold-free summary. Uncertainty is reported as paired-bootstrap 95% CIs; pairwise agreement/disagreement between systems is tested with McNemar. Latency is measured at true batch=1 (p50/p90), not batch-amortized.

We are explicit about statistical support: **only** the guard's pooled/in-house-held-out AUPRC, the guard-vs-Llama-Guard novel aggregate AUPRC, the base-vs-tuned F1 delta, and the guard-vs-GPT ΔF1 carry CIs. Baseline AUPRCs (ShieldGemma, Llama-Guard on the in-house pool) are point estimates. We therefore reserve the word "decisive" for the one comparison in which both sides carry CIs and they do not overlap (novel-set guard vs Llama-Guard); elsewhere we say "leads on point estimates."

We report several distinct F1 aggregations and define each to prevent conflation:

- **Pooled micro-F1 (0.794):** computed over all 2,018 in-house test rows pooled.
- **Macro-F1, in-dist (0.860):** unweighted mean of per-benchmark F1 over the four in-distribution benchmarks.
- **Macro-F1, in-house held-out (0.827):** unweighted mean of per-benchmark F1 over JailbreakBench and XSTest.
- **Base-vs-tuned aggregate F1 (tuned 0.794, base 0.713):** the base-vs-tuned decomposition (Section 5.5) uses these *same* 2,018 pooled rows, so the tuned aggregate equals the pooled micro-F1 above; base and tuned differ only by model (each at its own clean in-distribution calibration), not by row set.

3.9 Decontamination

Training data is leakage-filtered against the test splits. Our initial pipeline used exact-string-match decontamination. We recommend, and adopt as protocol, source-family holdout plus near-duplicate detection; source-family holdout is what defines and separates the four novel benchmarks, so the guard is evaluated on data from families it never saw during training. We describe near-duplicate detection as the recommended companion to family holdout rather than claiming an exhaustively executed near-duplicate sweep of the release pipeline.

Table 4: Benchmarks: source, safety axis, split role, and test statistics. Sources are Hugging Face datasets at `huggingface.co/datasets/⟨source⟩`. "Test n (uns:safe)" is the balanced test-split size and its unsafe:safe composition. The four in-distribution and two in-house-held-out sets form the 2,018-row in-house pool; the three balanced novel sets total 2,020 rows (per-set AUPRC in Table 7); HarmBench is 200 all-unsafe rows scored separately.

Benchmark	Source (Hugging Face)	Axis	Split role	Test n (uns:safe)
BeaverTails [16]	`PKU-Alignment/BeaverTails`	guardrail	in-dist (train/dev/test)	1,041 (513:528)
ToxicChat [19]	`lmsys/toxic-chat`	guardrail	in-dist (train/dev/test)	401 (199:202)
Prompt Injections	`deepset/prompt-injections`	red-team	in-dist (train/dev/test)	68 (32:36)
Jailbreak Classification	`jackhhao/jailbreak-classification`	red-team	in-dist (train/dev/test)	148 (73:75)
JailbreakBench [3]	`JailbreakBench/JBB-Behaviors`	red-team	in-house held-out (test only)	120 (58:62)
XSTest [27]	`natolambert/xstest-v2-copy`	over-refusal	in-house held-out (test only)	240 (123:117)
WildGuardTest [11]	`allenai/wildguardmix`	guardrail	novel / OOD (test only)	balanced
WildJailbreak [17]	`allenai/wildjailbreak`	red-team	novel / OOD (test only)	balanced
OR-Bench-Hard [4]	`bench-llm/or-bench`	over-refusal	novel / OOD (test only)	balanced
HarmBench [22]	`walledai/HarmBench`	red-team	novel / OOD (test only)	200 (all unsafe)

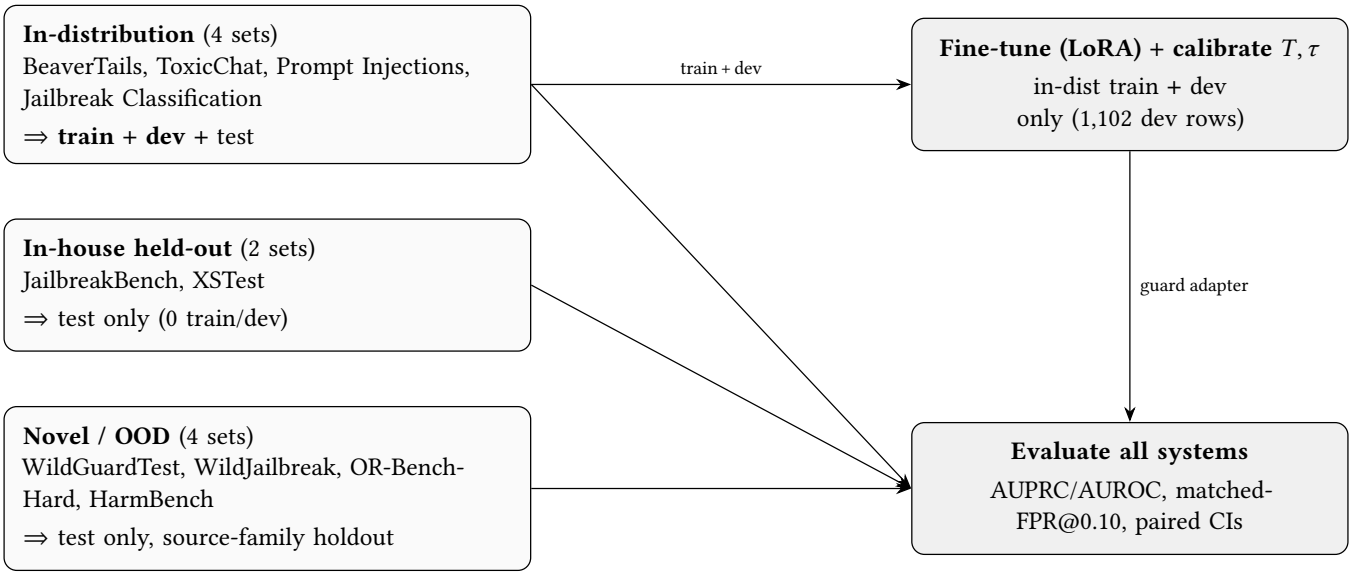

Figure 3: Benchmark pools and the train/dev/test separation. Only the four in-distribution sets feed fine-tuning and calibration (train + dev); the two in-house-held-out and four novel sets are test-only, so held-out and out-of-distribution evaluation measure transfer rather than memorization.

3.10 Evaluation bugs found and fixed

Building the pipeline surfaced four real bugs in our own first evaluation, each of which we corrected before reporting: (1) a **calibration leak** in which in-house held-out dev rows had leaked into temperature/threshold fitting — fixed to in-dist-only calibration ($T = 2.10$, $\tau = 0.59$); (2) **batch-amortized latency** (throughput/16) — replaced with true batch=1 p50/p90; (3) an **asymmetric threshold advantage** (only our model got a dev-tuned threshold while baselines were fixed points) — addressed via matched-FPR + threshold-free AUPRC; and (4) a **GPT error-handling bias** in which API errors defaulted to `unsafe` (inflating the GPT FPR) — corrected to *abstain*. We verified this does not affect the reported comparison: an abstention-logged re-score of the in-house GPT baseline found **zero** API errors in the original run (the fail-closed branch never fired), and it reproduces the tie (GPT FPR 0.321→0.326; guard − GPT ΔF1 +0.007, McNemar p=0.65).

4 IMPLEMENTATION FRAMEWORK

4.1 Hardware configuration

Training and evaluation run on a single consumer Apple M4 Max laptop (36 GB unified memory, MPS backend, no CUDA); everything — training, calibration, and evaluation — fits on the laptop. Guard inference latency at batch=1 is p50 124 ms / p90 188 ms.

4.2 Software and frameworks

The guard is built on Hugging Face `transformers` with `peft` (LoRA) and `trl`, PyTorch on the MPS backend, and `datasets` for benchmark loading; evaluation uses NumPy/pandas, and Matplotlib for figures. Exact pinned versions are released with the code (`requirements.txt`).

4.3 Model configuration and hyperparameters

Fine-tuning uses LoRA ($r = 32$, $\alpha = 64$, dropout 0.05) on all seven projection matrices (q, k, v, o, gate, up, down) with a completion-only objective on the single verdict token: 60.5M trainable parameters (1.93% of 3.14B), bf16, 300 optimizer steps in ≈71 minutes, seed 42. The full recipe is in Table 3 and the post-training calibration values in Table 2.

4.4 Reproducibility

All results are reproduced by the released scripts, orchestrated by a single notebook (`paper_reproduction.ipynb`) that runs each producing script and renders every table and figure from the emitted JSON, with a `verify_cached` mode (render committed artifacts) and a `recompute` mode (rerun scoring), plus a validation checklist against the reported numbers. Evaluation rows are reconstructed from the public benchmark datasets; we recommend pinning dataset revisions for byte-exact reruns.

4.5 System architecture

Figure 1 shows the end-to-end pipeline (data splits → fine-tuning and calibration → frozen evaluation) and Figure 2 the single-token logprob head that all systems are read through.

5 EXPERIMENTAL RESULTS AND ANALYSIS

We report all systems at threshold-free operating points (AUPRC/AUROC) and at a common, conservative operating point (matched-FPR@0.10), with paired-bootstrap 95% CIs where available. Latency is measured at true batch size 1.

5.1 Main comparison (in-house pool)

Threshold-free AUPRC here is pooled over the **in-house test rows only** (2,018 rows) — not over all evaluation rows, since ShieldGemma's novel runs are pending and the novel sets are reported as a separate pool in Section 5.2. Matched-FPR@0.10 fixes every tunable system to the same false-positive rate using a threshold set on the in-distribution dev split. gpt-5.4-mini is reported at its fixed native decision point (no tunable threshold), so it is not directly comparable at matched FPR.

On pooled in-house AUPRC the guard (0.844) leads ShieldGemma-2b (0.712) and Llama-Guard-3-1B (0.639) on point estimates (the baselines carry no CIs, so we do not assert statistical significance here); see Figure 4. At matched FPR@0.10 all tunable systems are conservative (the guard operates at recall 0.431, FPR 0.051); the guard still leads the other tunable guards on both recall and F1 on point estimates. The guard's in-dist AUPRC is 0.846 and its in-house held-out AUPRC is 0.860 [0.803, 0.909]. At each system's best threshold (Optimal-F1), the guard reaches 0.794, ShieldGemma-2b 0.730, and Llama-Guard-3-1B 0.672 — the same guard > ShieldGemma > Llama-Guard order, which (like AUPRC) inverts the native-threshold F1 ordering of the two open guards (Section 5.7).

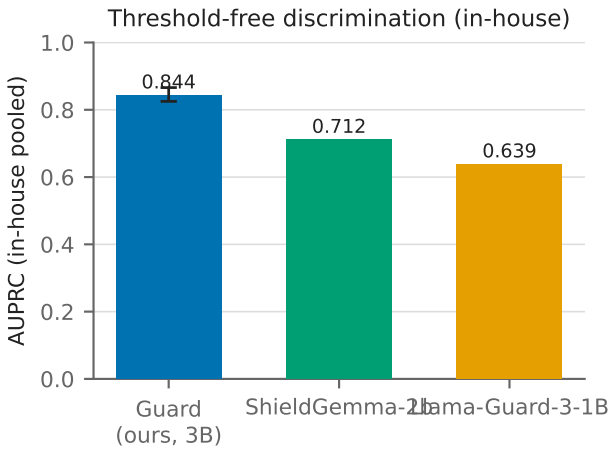

Figure 4: Pooled in-house AUPRC (2,018 test rows): the guard (0.844) leads ShieldGemma-2b (0.712) and Llama-Guard-3-1B (0.639) on point estimates.

Table 6 breaks the in-house pool down per benchmark: threshold-free AUPRC for every system, plus the base→tuned F1 at each model's own clean in-distribution calibration (the tuned column is also the guard's deployed-point F1). Two patterns stand out. Tuning's F1 gain is concentrated in-distribution and near-flat on the held-out sets (JailbreakBench +0.012, XSTest −0.023); and on the two held-out sets the *base* already has the higher AUPRC (JailbreakBench 0.967 vs 0.826, XSTest 0.976 vs 0.857), an early sign of the out-of-distribution ranking degradation we quantify in §5.5. At the deployed point ($T = 2.10$, $\tau = 0.59$) this yields macro-F1 0.860 (in-dist) / 0.827 (held-out) and pooled micro-F1 0.794, but the deployed overall FPR is **0.306** — at $\tau = 0.59$ roughly 31% of benign in-house prompts are flagged, a genuine deployment concern we revisit in Section 5.8.

5.2 Novel held-out benchmarks (guard never trained on them)

On the three balanced benchmarks the guard never saw during training, we compare threshold-free AUPRC across the tuned guard, the *zero-shot base*, and Llama-Guard-3-1B (both guard-vs-Llama and base-vs-guard aggregate CIs are non-overlapping).

The tuned guard's aggregate AUPRC (0.781 [0.751, 0.811]) exceeds Llama-Guard-3-1B's (0.701 [0.673, 0.733]) with non-overlapping CIs — a decisive cross-system result. But the *untuned base* ranks higher still (0.886 [0.870, 0.900], disjoint from the tuned guard), while best-threshold F1 is essentially tied across base and guard (0.792 vs 0.796, both above Llama-Guard's 0.691); see Figure 5. Read together: out-of-distribution the base is the strongest *ranker* (most operating-point-robust), base and tuned guard reach the same *peak* F1, and both beat Llama-Guard. The guard's novel-set lead over Llama-Guard is thus inherited from

Table 5: Main comparison on the in-house pool (2,018 balanced test rows). AUPRC is threshold-free; recall/F1/FPR are reported at matched-FPR@0.10; gpt-5.4-mini is shown at its fixed native decision point. Only the guard carries CIs; baseline AUPRCs are point estimates.

System	AUPRC, in-house pooled [95% CI]	Recall @FPR 0.10	F1 @FPR 0.10	FPR @0.10	Latency (batch=1)
Guard (SmolLM3-3B, ours)	**0.844** [0.825, 0.866]	0.431	0.581	0.051	p50 124 ms / p90 188 ms
ShieldGemma-2b	0.712	0.341	0.464	—	not measured
Llama-Guard-3-1B	0.639	0.242	0.360	—	not measured
gpt-5.4-mini (fixed native point)	—	0.856	0.784	0.321	~512 ms (remote API)
keyword matcher	—	0.096	0.168	—	not measured

Table 6: Per-benchmark in-house results. Threshold-free AUPRC (unsafe = positive) for each system, and base→tuned F1 at each model's clean in-distribution calibration. Sets marked † are in-house held-out (test-only, zero train/dev rows); the rest are in-distribution. "n (uns:safe)" is the balanced test size. Novel per-set AUPRC is in Table 7.

Benchmark	n (uns:safe)	Guard	Base	Llama-G-3-1B	ShieldGemma-2b	F1 base→tuned
BeaverTails	1,041 (513:528)	0.687	0.618	0.561	0.656	0.691 → 0.716
ToxicChat	401 (199:202)	0.948	0.856	0.663	0.952	0.766 → 0.915
Prompt Injections	68 (32:36)	0.905	0.786	0.807	0.666	0.690 → 0.828
Jailbreak Classification	148 (73:75)	**0.999**	0.451	0.662	0.824	0.339 → 0.980
JailbreakBench†	120 (58:62)	0.826	**0.967**	0.806	0.676	0.809 → 0.821
XSTest†	240 (123:117)	0.857	**0.976**	0.755	0.825	0.856 → 0.833
Pooled (2,018)	(998:1,020)	**0.844**	0.696	0.639	0.712	0.713 → 0.794

Table 7: Threshold-free AUPRC on the novel held-out benchmarks. The untuned base outranks the tuned guard, which in turn beats Llama-Guard-3-1B; the aggregate is over the three balanced sets (2,020 rows) and excludes HarmBench.

Benchmark	Base (zero-shot) AUPRC	Guard (tuned) AUPRC	Llama-Guard-3-1B AUPRC
wildguardtest	0.894	0.798	0.751
wildjailbreak	0.837	0.722	0.597
orbench_hard	0.940	0.845	0.750
Aggregate (2,020 rows)	**0.886** [0.870, 0.900]	**0.781** [0.751, 0.811]	**0.701** [0.673, 0.733]
Aggregate Optimal-F1	0.792	0.796	0.691

the base, not produced by LoRA; fine-tuning degrades OOD score ranking (AUPRC) without changing peak achievable F1 (Section 5.5).

The aggregate 95% CIs ([0.751, 0.811] vs [0.673, 0.733]) do not overlap, so this is our one decisive cross-system AUPRC result. The aggregate is over the three balanced novel sets (2,020 rows); **HarmBench is excluded** from it. On HarmBench (200 all-harmful prompts), mean predicted $P(\text{unsafe})$ is 0.967 for the zero-shot base, 0.780 for the tuned guard, and 0.390 for Llama-Guard-3-1B; because this compares raw probabilities across differently temperature-scaled models it is *suggestive, not decisive* and is not a threshold-free ranking metric (though it echoes the AUPRC ordering base > guard > Llama-Guard). ShieldGemma-2b on the novel benchmarks is **pending** (Gemma-2 stalls on the throttled MPS laptop); on the in-house held-out set its AUPRC is 0.785 versus the guard's 0.860. We therefore make **no** ShieldGemma novel-set claim.

Table 8: Guard versus gpt-5.4-mini on paired rows: a statistical tie on F1 at roughly 4x lower batch=1 latency.

Comparison (guard vs gpt-5.4-mini)	Value
ΔF1 (guard − gpt)	+0.010
95% CI	[−0.006, 0.027]
McNemar p	0.20
Guard latency (batch=1)	124 ms (local M4 Max)
gpt-5.4-mini latency	~512 ms (remote API)

5.3 Frontier comparison: statistical tie and Pareto-style position

Against gpt-5.4-mini on paired rows, the guard shows no statistically significant F1 difference.

The F1 difference is not significant (McNemar $p = 0.20$; CI spans zero, its lower bound admitting the guard being 0.006 *worse*), so this is a statistical **tie** with the guard nominally ahead, not an accuracy beat. Two caveats. First, this comparison places the guard

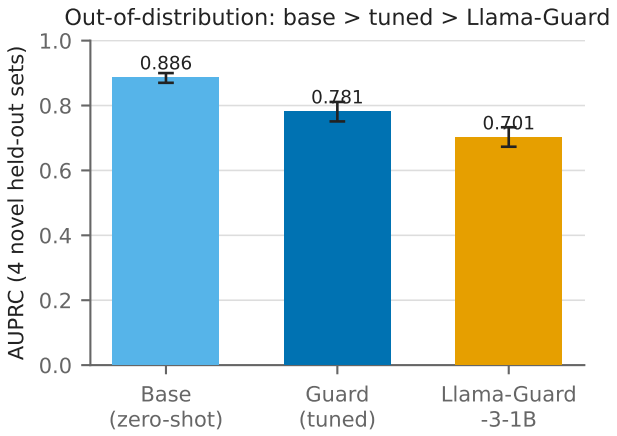

Figure 5: Aggregate AUPRC on the novel held-out benchmarks (2,020 balanced rows): the zero-shot base (0.886) outranks the tuned guard (0.781), which in turn beats Llama-Guard-3-1B (0.701), with non-overlapping 95% CIs.

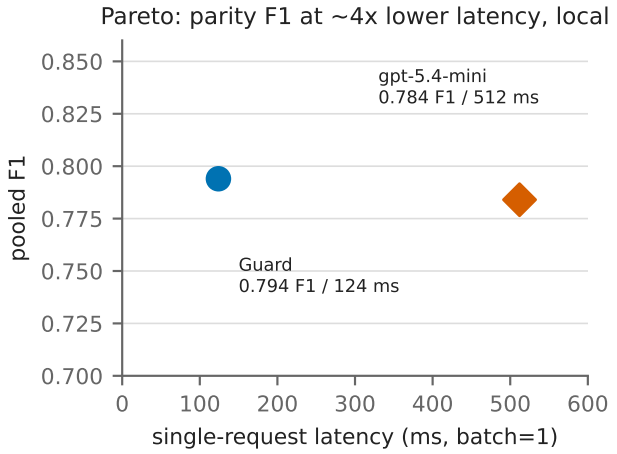

Figure 6: F1 versus single-request (batch=1) latency: the guard sits at parity with gpt-5.4-mini on F1 at roughly 4x lower latency (124 ms vs ~512 ms per request).

at its in-dist-tuned threshold ($\tau = 0.59$) against gpt at its fixed native point — the same per-model-threshold asymmetry we criticize in Section 5.7; because gpt exposes no tunable threshold, a matched-FPR comparison is not possible, so the tie is threshold-advantaged in the guard's favor and should be read as such. Second, latency is cross-substrate: 124 ms is local M4 Max inference while ~512 ms is a remote API round-trip (network plus server-side batching), so "~4x lower latency" is deployment-realistic but *not* a controlled hardware comparison, and it is a **guard-vs-gpt claim only** — Llama-Guard and ShieldGemma latency were not measured. With those caveats, the guard achieves parity accuracy locally at roughly 4x lower batch=1 latency with no API cost: an

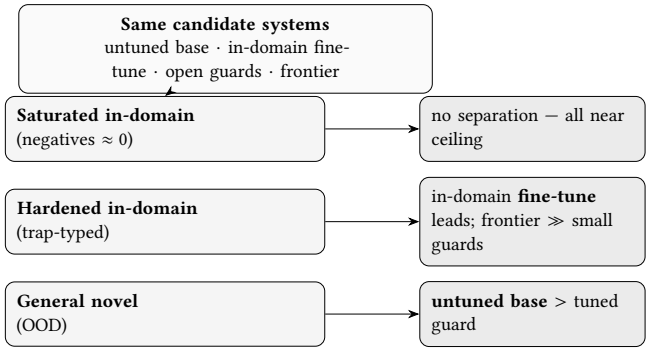

Figure 7: The benchmark chooses the winner. The identical systems yield a different verdict per regime: a saturated benchmark separates nobody; a hardened in-domain rebuild is led by the in-domain fine-tune, with a frontier model far ahead of every small guard; and on general out-of-distribution benchmarks the untuned base outranks the tuned guard (§5.5, §5.6).

efficiency win at parity accuracy rather than an accuracy win (Figure 6).

5.4 Analysis: why the rankings move

The three analyses below share one message, summarized in Figure 7: the *same* candidate systems change rank as the evaluation regime changes, so the reported "best guard" is a property of the benchmark and operating point as much as of the model.

5.5 Base-vs-tuned decomposition: PEFT specializes in-distribution and degrades OOD ranking

We isolate the contribution of parameter-efficient fine-tuning by evaluating the base SmolLM3-3B model zero-shot with the identical single-token logprob head (no training), on the identical paired rows.

Two findings stand out. First, the *untuned* base is already a competent safety classifier by F1: its zero-shot F1 of 0.713 exceeds the zero-shot F1 of both purpose-built open guards — Llama-Guard-3-1B (0.673) and ShieldGemma-2b (0.424). Consistent with our own Contribution #1, this is an **F1-at-operating-point** comparison, so we also measured the base's threshold-free in-house AUPRC: 0.696 [0.666, 0.730]. This *confirms* the caution we flagged rather than overturning it — under AUPRC the base (0.696) falls *between* Llama-Guard-3-1B (0.639) and ShieldGemma-2b (0.712), so the base's by-F1 lead over ShieldGemma does **not** survive the threshold-free test in-house. The threshold-free "base is a strong guard" claim holds only *out-of-distribution*: on the four novel benchmarks the base's AUPRC (0.886) beats every system we could measure there (Llama-Guard-3-1B 0.701; see Section 5.2).

Second, LoRA adds a real but *localized* gain: aggregate F1 on the identical 2,018 paired rows (each model at its own clean in-distribution calibration) rises from 0.713 to 0.794 (delta +0.081, 95% CI [0.062, 0.100], McNemar $p < 1e{-}4$). Decomposing this delta per benchmark shows it is concentrated almost entirely

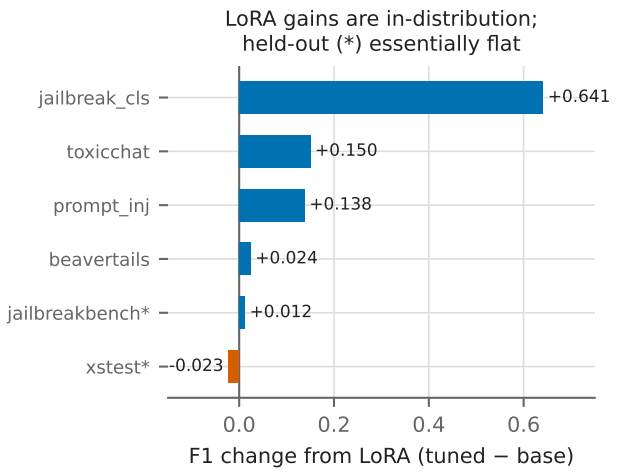

Figure 8: Per-benchmark base → tuned F1 delta. LoRA gains are positive in-distribution and flat-to-negative on the in-house held-out benchmarks.

in the in-distribution axes and is essentially flat on the two in-house held-out benchmarks (JailbreakBench +0.012, XSTest −0.023; macro −0.005). The threshold-free picture is symmetric and sharper: tuning *raises* the guard's in-house AUPRC from the base's 0.696 to 0.844, yet *lowers* its novel-set AUPRC from the base's 0.886 to 0.781 — LoRA specializes the ranker to the in-distribution regime at the cost of out-of-distribution ranking.

The base's in-house shortfall is concentrated, not uniform. Per-benchmark, the base's low in-house AUPRC (0.696) comes almost entirely from BeaverTails — where the zero-shot base over-flags edgy-but-benign prompts (AUPRC 0.618, and this set is 52% of the pool) — and from jailbreak_classification, which the zero-shot base *inverts* (0.451, below chance); on the other four in-house benchmarks and all three novel sets the base already ranks well (0.79–0.98). Tuning's in-house AUPRC lift is therefore mostly the repair of these two weak spots (jailbreak_classification 0.451 → 0.999) plus improved score comparability across benchmarks — the pooled-versus-macro-mean gap shrinks from 0.080 (base 0.696 vs 0.776) to 0.026 (tuned 0.844 vs 0.870) — rather than a broad gain in discrimination. This also explains the base's counterintuitive in-house-below-novel ordering: it is a benchmark-composition effect (the in-house pool contains the base's two blind spots; the novel pool does not), not evidence that the base generalizes better to unseen data than to seen data.

The pattern is unambiguous within its scope: every large positive delta is in-distribution (the extreme case, jailbreak_classification, +0.641, is a task the base essentially could not do), while the two in-house held-out benchmarks barely move (JailbreakBench +0.012, XSTest −0.023; macro −0.005), including a small regression on the over-refusal probe (XSTest). Figure 8 visualizes this split. LoRA specializes the guard to the training distribution's label conventions and formatting; on the two in-house held-out benchmarks it does not measurably improve — and on over-refusal can mildly degrade — generalization.

We have now measured the base on the four novel benchmarks, which **confirms and sharpens** this claim. Out-of-distribution, the *untuned* base ranks higher than the tuned guard (aggregate AUPRC 0.886 [0.870, 0.900] vs 0.781 [0.751, 0.811], non-overlapping CIs), while both reach essentially the same best-threshold F1 (0.792 vs 0.796). Fine-tuning therefore does not merely fail to help OOD — it *degrades the guard's OOD score ranking / operating-point robustness*: a lower AUPRC means a worse precision–recall trade-off across thresholds, even though a per-distribution re-tuned threshold would recover the same peak F1 (a re-tuning that is, by definition, unavailable on unseen data). This resolves the earlier tension between Contribution #2 (tuning does not improve — and on OOD ranking actively harms — generalization) and Contribution #3 (the tuned guard still beats Llama-Guard on novel data): the guard's novel-set lead is *inherited from the base*, not produced by LoRA, and the base in fact beats Llama-Guard by an even larger margin.

This is the constructive complement to prior reports that fine-tuning can *destroy* safety alignment: here tuning "works," yet the measurable gain is memorization of the in-distribution operating regime rather than demonstrated transferable safety competence.

5.6 Case study: a saturated domain benchmark and its hardened rebuild

The base-vs-tuned finding above is not specific to our novel sets: the winner also moves when we control a benchmark's difficulty directly. Take a domain guard task — mortgage fair-lending, compliance, and security-misuse prompts. The obvious benchmark is *saturated*: the flag and allow classes are lexically separable (overt violations vs. clean textbook questions) and the negatives sit near probability zero, so a floor threshold trivially divides them and every system — base, in-domain fine-tune, and open guards alike — scores near the ceiling and is statistically indistinguishable. A saturated set cannot rank guards, so we replace it with a hardened one.

The hardened set[1] contains 334 trap-typed items (195 unsafe / 139 safe; 284 *very-hard* and 50 *borderline*; loan-officer and consumer personas) engineered to defeat surface cues (Table 10). It mixes disguised *positives* — minimal-pair twins, euphemism, coded-proxy, buried instruction-injection, dual-use, and multi-turn setups — with hard *negatives*: benign prompts that look risky, such as fair-lending education questions, legitimate business-justified underwriting, and over-refusal bait. The set has no train/dev split, so we report operating-point-independent AUPRC and Optimal-F1 as the fair primary, plus accuracy, recall, precision, and FPR at each system's own best-F1 point (gpt-5.4-mini at its native point).

Three things follow (Table 11). First, the hardened set **de-saturates** the comparison: where the saturated version left every system near the ceiling, here threshold-free AUPRC spreads from 0.82 to 0.92 and accuracy from 0.65 to 0.95. Second, the difficulty is **over-blocking, not misses**: every small guard keeps high recall on the disguised positives (0.93–0.95; minimal-pair and

[1]Released in the artifact bundle as `guard_benchmark_hard.jsonl` (334 items) and scored by `eval_mortgage_hard.py`; the saturated legacy set `guard_benchmark.jsonl` is released alongside. Full paths are in the repository README.

Table 9: Per-benchmark base-vs-tuned F1 decomposition on the identical 2,018 in-house test rows, each model at its own clean in-distribution calibration (tuned T=2.10, τ=0.59; base T=7.00, τ=0.51). Positive deltas are concentrated in-distribution; the two in-house held-out benchmarks move only marginally (macro −0.005).

Benchmark	Split	Base F1	Tuned F1	Delta
jailbreak_classification	in-dist	0.339	0.980	+0.641
toxicchat	in-dist	0.766	0.915	+0.150
prompt_injections	in-dist	0.690	0.828	+0.138
beavertails	in-dist	0.691	0.716	+0.024
jailbreakbench	in-house held-out	0.809	0.821	+0.012
xstest	in-house held-out	0.856	0.833	−0.023
Aggregate (paired rows)		**0.713**	**0.794**	+0.081 **[0.062, 0.100]**

Table 10: Composition of the hardened mortgage set (334 items). "Trap type" is the surface cue an item is built to defeat; *hard_negative* items are benign prompts that superficially resemble violations. Also: 195 unsafe / 139 safe; difficulty very-hard 284 / borderline 50; categories safe 139, security-misuse 89, fair-lending 65, compliance 41.

Trap type	n
hard_negative (benign, looks risky)	75
minimal_pair	60
euphemism	52
business_justified	45
buried_injection	39
over_refusal_bait	20
dual_use	15
multi_turn	15
coded_proxy	13

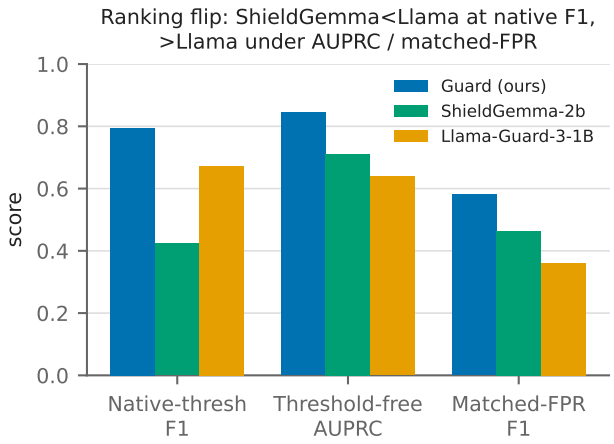

Figure 9: ShieldGemma-2b vs Llama-Guard-3-1B under native-threshold F1, threshold-free AUPRC, and matched-FPR@0.10: the native-F1 ranking inverts under both operating-point-fair metrics.

coded-proxy catch-rates ≈1.0) but pays a steep false-positive rate on the hard negatives (base 0.53, general-safety guard 0.74, mortgage-SFT 0.27). The frontier gpt-5.4-mini sits at FPR 0.065 for comparable recall (0.96). We stress this last comparison is *not* a matched operating point — each local guard is read at its own best-F1 threshold and gpt at its fixed native point — so it is suggestive of a small-guard-vs-frontier capability gap rather than a controlled one; the threshold-free AUPRC below is the claim we lean on. Third, the **winner is benchmark-dependent** (on the fair, threshold-free AUPRC): the in-domain mortgage-SFT — indistinguishable on the saturated set — has the highest AUPRC here (0.924), ahead of the untuned base (0.892; the gap is within overlapping CIs) and clearly ahead of the general-safety guard (0.824). This is the *opposite* in-domain-tuning verdict from the general novel benchmarks (§5.5), where the untuned base outranks the tuned guard — so whether in-domain fine-tuning helps depends on whether the hard test is in-domain or general OOD.

We present this as a case study, not a headline benchmark: the hardened set is seed-scale (334 items) and single-annotator, and because every system keeps high recall the discriminating signal lives in AUPRC and FPR rather than recall. Absolute magnitudes will move under broader adjudication; the robust reading is comparative — the de-saturation, the small-guard-vs-frontier FPR gap, and the benchmark-dependent fine-tuning verdict (in-domain tuning leads *here* but the untuned base leads on general OOD) — all instances of the benchmark choosing the winner.

5.7 Operating-point fairness: rankings depend on the threshold

Guard rankings shift substantially with the decision threshold, so any comparison that reads F1 at each model's *native* operating point conflates classifier quality with operating-point choice. Concretely, at native thresholds Llama-Guard-3-1B posts a higher F1 than ShieldGemma-2b (0.673 vs 0.424), but this ordering **reverses** under both threshold-free AUPRC (0.639 vs 0.712) and best-threshold Optimal-F1 (0.672 vs 0.730): a ranking flip that is purely an artifact of where each model's alarm is set. Figure 9 shows this flip. Only our model received a dev-tuned threshold; the baselines were fixed points. We therefore report both threshold-free AUPRC and a matched-FPR comparison, with paired-bootstrap CIs where available.

Table 11: Results on the hardened mortgage benchmark (334 items). AUPRC and Optimal-F1 are threshold-free / operating-point-independent (fair primary); accuracy, recall, precision, and FPR are at each system's own best-F1 point, with gpt-5.4-mini at its native decision point. Every small guard over-blocks the hard negatives (FPR 0.27–0.74) where the frontier holds 0.065; the in-domain fine-tune is the strongest small guard.

System	AUPRC [95% CI]	Opt-F1	Accuracy	Recall	Precision	FPR
Base (zero-shot SmolLM3-3B)	0.892 [0.854, 0.925]	0.819	0.754	0.954	0.718	0.525
Mortgage-SFT (in-domain LoRA)	0.924 [0.849, 0.946]	0.885	0.856	0.949	0.830	0.273
General-safety guard (ours)	0.824 [0.774, 0.873]	0.756	0.650	0.928	0.637	0.741
gpt-5.4-mini (frontier ceiling)	—	—	**0.949**	**0.959**	**0.954**	**0.065**

Table 12: Threshold-free AUPRC across in-house poolings. The guard leads on point estimates; only the guard carries CIs.

System	AUPRC in-house pooled	AUPRC in-dist	AUPRC in-house held-out
Guard	0.844 [0.825, 0.866]	0.846	0.860 [0.803, 0.909]
ShieldGemma-2b	0.712	0.702	0.785
Llama-Guard-3-1B	0.639	0.610	0.762

Table 13: Dev-matched FPR=0.10 operating point (threshold set so FPR=0.10 on the in-distribution dev split; realized *test* FPRs differ and are shown in the FPR column). gpt-5.4-mini and the keyword matcher are at their fixed native points.

System	Recall	F1	FPR
Guard	0.431	0.581	0.051
ShieldGemma-2b	0.341	0.464	0.125
Llama-Guard-3-1B	0.242	0.360	0.101
keyword matcher	0.096	0.168	0.047
gpt-5.4-mini (fixed native point)	0.856	0.784	0.321

Threshold-free, the guard leads the open guards across every in-house pooling, on point estimates (baseline AUPRCs carry no CIs):

At a conservative operating point (each tunable system's threshold set so its FPR=0.10 on the in-distribution *dev* split), the ranking is preserved on point estimates but all systems are pushed to low recall. The realized *test* FPRs are not identical — guard 0.051, Llama-Guard-3-1B 0.101, ShieldGemma-2b 0.125 — so "matched-FPR@0.10" is matched on the *dev* split; on the test pool the guard is in fact held to a *stricter* operating point than the baselines, which disadvantages rather than flatters it:

Why native-threshold F1 comparisons mislead: gpt-5.4-mini's native point trades a high FPR (0.321) for high recall (0.856), producing an F1 of 0.784 that is not comparable to a guard evaluated at FPR 0.051. Conversely, the matched-FPR@0.10 constraint makes *every* system conservative — the guard's recall there is only 0.431 — so matched-FPR alone understates absolute capability. The two views are complementary: matched-FPR fixes the confound of who got to tune a threshold, and AUPRC is the threshold-free summary that removes operating-point choice entirely. Read together they give a consistent picture on point estimates — guard > ShieldGemma-2b > Llama-Guard-3-1B — with the one formally supported (non-overlapping-CI) statement being the guard's novel-set lead over Llama-Guard (see Section 5.2).

5.8 Limitations

We frame these results as a measurement and controlled study, not a state-of-the-art claim. Several caveats bound their interpretation.

Deployed operating point over-blocks. At the deployed point ($T = 2.10$, $\tau = 0.59$) the guard's overall FPR is 0.306 — roughly 31% of benign in-house prompts are flagged `unsafe`. This is a real deployment concern: a global τ tuned for balanced F1 on in-dist dev data is too aggressive for many production settings, which would need a higher, application-specific threshold (trading recall for fewer false positives). We report the deployed point for transparency, but we do not recommend $\tau = 0.59$ as an out-of-the-box safe default.

Operating point and matched-FPR conservatism. A single global threshold is structurally in-distribution-optimized, and guard rankings depend heavily on the decision threshold. Our matched-FPR@0.10 comparison forces all systems into a conservative regime: at that operating point the guard's recall is only 0.431 (F1 0.581, FPR 0.051). We therefore treat threshold-free AUPRC as the fair primary summary and matched-FPR as a secondary, deliberately strict view.

Statistical support is uneven. Only four comparisons carry CIs (guard pooled/in-house-held-out AUPRC, guard-vs-Llama novel aggregate, base-vs-tuned F1 delta, guard-vs-GPT ΔF1). Baseline AUPRCs are point estimates, so the pooled/in-dist "leads" and the matched-FPR ranking are point-estimate claims, not significance tests. We reserve "decisive" for the single non-overlapping-CI result (novel guard vs Llama-Guard).

GPT is a tie, not a beat, and threshold-advantaged. Against gpt-5.4-mini the guard shows no significant F1 difference (Table 8); the comparison uses the guard's dev-tuned threshold against gpt's fixed native point (no matched-FPR comparison is possible), and the latency edge is cross-substrate (local vs remote API) — an efficiency, not accuracy, advantage.

Novel-set win is inherited, not tuned. The guard's novel-set lead over Llama-Guard reflects inherited base competence, not LoRA — the untuned base outranks the tuned guard OOD (Section 5.5). The result is anchored on operating-point-independent AUPRC and Optimal-F1, so it does not depend on any calibration choice.

Generalization claim rests on two benchmarks. The "PEFT specializes but does not improve generalization" finding is evidenced by only the two in-house held-out benchmarks (JailbreakBench, XSTest); it is not a broad OOD-generalization result.

ShieldGemma out-of-regime and pending on the novel set. ShieldGemma-2b is designed for content-policy / general-guideline moderation and is not an injection detector; using it across our red-team axes is somewhat out of its designed regime, so its numbers may understate its intended performance. On the in-house held-out set its AUPRC is 0.785 vs the guard's 0.860. Its results on the four novel held-out benchmarks are **pending** (Gemma-2 stalls on the throttled MPS laptop) and are not reported here; we make no ShieldGemma novel-set claim.

HarmBench metric is calibration-dependent. The HarmBench comparison (mean $P(\text{unsafe})$ 0.780 vs 0.390) compares raw probabilities across differently temperature-scaled models; it is suggestive, not decisive, and HarmBench is excluded from the aggregate AUPRC.

Dataset licensing. ToxicChat is non-commercial; it is eval-only for any released model and cannot be used to train a distributed guard.

Sample sizes and CIs. Some benchmarks have small n and correspondingly wide CIs, so per-benchmark point estimates should be read with their intervals (e.g., in-house held-out AUPRC 0.860 [0.803, 0.909]).

Decontamination scope. We adopt source-family holdout to define the novel benchmarks and recommend near-duplicate detection as the companion protocol; we do not claim an exhaustive near-duplicate sweep of the entire release pipeline.

Scope of the guard and evaluation. The guard emits a single-token verdict from one forward pass; we do not evaluate a multi-token or reasoning-style guard, and adversarial-robustness evaluation (e.g., against adaptive attacks) is future work.

6 CONCLUSION AND FUTURE WORK

We presented a binary *input-prompt* safety guard — the prompt-classification component of the AGENT-BOUNCER project — obtained by parameter-efficient fine-tuning of SmolLM3-3B on a single consumer laptop (Apple M4 Max, MPS, no CUDA). The guard emits a single-token verdict in one forward pass. Our central finding is a measurement one: guard rankings depend heavily on the decision threshold, so we evaluate at matched FPR, with threshold-free AUPRC/AUROC, paired-bootstrap 95% CIs, and McNemar tests. Building the pipeline surfaced and let us fix real evaluation bugs (calibration leak, batch-amortized latency, per-model-only threshold tuning, GPT error-to-"unsafe" default, exact-string decontamination).

Under this protocol the results are modest and specific rather than sweeping (headline numbers in Table 1). On threshold-free AUPRC the guard surpasses Llama-Guard-3-1B decisively — the only non-overlapping-CI cross-system result, and it holds on the four novel held-out benchmarks — and it exceeds ShieldGemma-2b on the in-house pooled / in-dist / in-house-held-out sets on point estimates (ShieldGemma's novel runs are pending). It is a statistical tie with gpt-5.4-mini on F1 at ~4x lower batch=1 latency and no API cost: an efficiency win at parity accuracy, not an accuracy beat.

Our base-vs-tuned decomposition tempers the fine-tuning narrative (numbers in Tables 7 and 9). The zero-shot base is already a competent classifier, and LoRA's gains are almost entirely in-distribution: the two in-house held-out benchmarks barely move, and on the four novel benchmarks the *untuned* base actually outranks the tuned guard (aggregate AUPRC 0.886 vs 0.781, disjoint CIs) at tied peak F1. The guard's out-of-distribution lead is therefore inherited base competence, and tuning *degrades* out-of-distribution ranking rather than improving generalization. The mortgage case study (§5.6) makes the point in a domain: a saturated benchmark ranks every system near the ceiling, while a hardened, trap-typed rebuild de-saturates the field and exposes a small-guard-vs-frontier over-blocking gap (benign FPR 0.27–0.74 vs. 0.065) — and the fine-tuning verdict flips (in-domain tuning leads there, the untuned base leads on general OOD). The recurring lesson is that a guard's apparent quality is inseparable from the operating point and the benchmark it is measured on.

6.1 Future Work

- **Specializing without sacrificing OOD robustness.** Since fine-tuning specialized in-distribution but *degraded* the base's OOD score ranking (Section 5.5), a central direction is training recipes — broad, family-decontaminated data plus regularization — that add in-distribution competence *without* losing the untuned base's operating-point robustness on unseen distributions.
- **Broad-data training as the generalization lever.** Since tuning specialized in-distribution without improving the in-house held-out F1, the most promising direction is scaling and diversifying training data (source-family coverage, near-duplicate-filtered) to test whether generalization, not just in-distribution fit, can be trained in.
- **Controlled objective comparison.** A like-for-like head-to-head of SFT vs DPO [26] vs IPO [1] vs KTO [6] vs ORPO [12] (and single-token GRPO [29] vs PPO [28]) under identical data, splits, and calibration, to isolate the effect of the learning objective on both in-distribution specialization and held-out transfer.
- **Hardened, expert-labeled benchmarks.** The mortgage case study (Section 5.6) used a seed-scale, single-annotator hardened set; expert re-adjudication, scaling, and applying the same saturated-vs-hardened rebuild across domains would turn the observations — de-saturation and the small-guard-vs-frontier over-blocking gap — from a case study into a headline result, and provide a benchmark whose difficulty is calibrated to model capability.
- **Adversarial robustness.** Evaluation and hardening against paraphrase, obfuscation, and prompt-level attacks, which our current single-token guard does not measure.
- **Agent surfaces (the namesake, not yet evaluated).** Extending the guard beyond input prompts to tool-call arguments, tool outputs, and model responses — the surfaces that motivate the AGENT-BOUNCER name but that this paper does *not* evaluate. This is the highest-value next step

and the most direct way to make the "agent" framing empirical rather than aspirational.

- **Completing the baseline matrix.** Resolving the pending ShieldGemma-2b runs on the novel held-out benchmarks, and measuring open-guard latency, for a complete threshold-free and efficiency comparison.

Code, calibration, and the full evaluation protocol are released as open code — the evaluation scripts plus a single orchestrating reproduction notebook (`paper_reproduction.ipynb`) — to support reproduction at laptop scale.

Baseline note: `gpt-5.4-mini` *is a hosted commercial API model with no public paper or model card; we cite it as an API baseline rather than a bibliographic reference.*

REFERENCES

[1] Mohammad Gheshlaghi Azar, Mark Rowland, Bilal Piot, Daniel Guo, Daniele Calandriello, Michal Valko, and Rémi Munos. 2023. A General Theoretical Paradigm to Understand Learning from Human Preferences. In *AISTATS 2024*. arXiv:2310.12036

[2] Elie Bakouch, Loubna Ben Allal, Anton Lozhkov, Nouamane Tazi, Lewis Tunstall, Carlos Miguel Patiño, Edward Beeching, Aymeric Roucher, Aksel Joonas Reedi, Quentin Gallouédec, Kashif Rasul, Nathan Habib, Clémentine Fourrier, Hynek Kydlíček, Guilherme Penedo, Hugo Larcher, Mathieu Morlon, Vaibhav Srivastav, Joshua Lochner, Xuan-Son Nguyen, Colin Raffel, Leandro von Werra, and Thomas Wolf. 2025. SmolLM3: smol, multilingual, long-context reasoner. Hugging Face blog + model card (HuggingFaceTB/SmolLM3-3B), https://huggingface.co/blog/smollm3. no arXiv paper; cite blog + model card.

[3] Patrick Chao, Edoardo Debenedetti, Alexander Robey, Maksym Andriushchenko, Francesco Croce, Vikash Sehwag, Edgar Dobriban, Nicolas Flammarion, George J. Pappas, Florian Tramèr, Hamed Hassani, and Eric Wong. 2024. JailbreakBench: An Open Robustness Benchmark for Jailbreaking Large Language Models. In *NeurIPS 2024 (Datasets & Benchmarks)*. arXiv:2404.01318

[4] Justin Cui, Wei-Lin Chiang, Ion Stoica, and Cho-Jui Hsieh. 2025. OR-Bench: An Over-Refusal Benchmark for Large Language Models. In *ICML 2025*. arXiv:2405.20947

[5] Yihe Deng, Yu Yang, Junkai Zhang, Wei Wang, and Bo Li. 2025. DuoGuard: A Two-Player RL-Driven Framework for Multilingual LLM Guardrails. *arXiv preprint* (2025). arXiv:2502.05163

[6] Kawin Ethayarajh, Winnie Xu, Niklas Muennighoff, Dan Jurafsky, and Douwe Kiela. 2024. KTO: Model Alignment as Prospect Theoretic Optimization. In *ICML 2024*. arXiv:2402.01306

[7] Igor Fedorov, Kate Plawiak, Lemeng Wu, Tarek Elgamal, Naveen Suda, Eric Smith, Hongyuan Zhan, Jianfeng Chi, Yuriy Hulovatyy, Kimish Patel, Zechun Liu, Changsheng Zhao, Yangyang Shi, Tijmen Blankevoort, Mahesh Pasupuleti, Bilge Soran, Zacharie Delpierre Coudert, Rachad Alao, Raghuraman Krishnamoorthi, and Vikas Chandra. 2024. Llama Guard 3-1B-INT4: Compact and Efficient Safeguard for Human-AI Conversations. *arXiv preprint* (2024). arXiv:2411.17713

[8] Shaona Ghosh, Prasoon Varshney, Erick Galinkin, and Christopher Parisien. 2024. AEGIS: Online Adaptive AI Content Safety Moderation with Ensemble of LLM Experts. *arXiv preprint* (2024). arXiv:2404.05993

[9] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. 2017. On Calibration of Modern Neural Networks. In *ICML 2017*. arXiv:1706.04599

[10] William Hackett, Lewis Birch, Stefan Trawicki, Neeraj Suri, and Peter Garraghan. 2025. Bypassing LLM Guardrails: An Empirical Analysis of Evasion Attacks against Prompt Injection and Jailbreak Detection Systems. In *First Workshop on LLM Security (LLMSEC 2025), ACL Anthology 2025.llmsec-1.8*. arXiv:2504.11168 v3 title; v1/v2 titled "Bypassing Prompt Injection and Jailbreak Detection in LLM Guardrails".

[11] Seungju Han, Kavel Rao, Allyson Ettinger, Liwei Jiang, Bill Yuchen Lin, Nathan Lambert, Yejin Choi, and Nouha Dziri. 2024. WildGuard: Open One-Stop Moderation Tools for Safety Risks, Jailbreaks, and Refusals of LLMs. In *NeurIPS 2024 (Datasets & Benchmarks)*. arXiv:2406.18495

[12] Jiwoo Hong, Noah Lee, and James Thorne. 2024. ORPO: Monolithic Preference Optimization without Reference Model. In *EMNLP 2024*. arXiv:2403.07691

[13] Lei Hsiung, Tianyu Pang, Yung-Chen Tang, Linyue Song, Tsung-Yi Ho, Pin-Yu Chen, and Yaoqing Yang. 2025. Why LLM Safety Guardrails Collapse After Fine-tuning: A Similarity Analysis Between Alignment and Fine-tuning Datasets. In *ICML 2025 DIG-BUGS workshop (also ACL Anthology 2026)*. arXiv:2506.05346

[14] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2021. LoRA: Low-Rank Adaptation of Large Language Models. In *ICLR 2022*. arXiv:2106.09685

[15] Hakan Inan, Kartikeya Upasani, Jianfeng Chi, Rashi Rungta, Krithika Iyer, Yuning Mao, Michael Tontchev, Qing Hu, Brian Fuller, Davide Testuggine, and Madian Khabsa. 2023. Llama Guard: LLM-based Input-Output Safeguard for Human-AI Conversations. *arXiv preprint* (2023). arXiv:2312.06674

[16] Jiaming Ji, Mickel Liu, Juntao Dai, Xuehai Pan, Chi Zhang, Ce Bian, Ruiyang Sun, Boyuan Chen, Yizhou Wang, and Yaodong Yang. 2023. BeaverTails: Towards Improved Safety Alignment of LLM via a Human-Preference Dataset. In *NeurIPS 2023 (Datasets & Benchmarks)*. arXiv:2307.04657

[17] Liwei Jiang, Kavel Rao, Seungju Han, Allyson Ettinger, Faeze Brahman, Sachin Kumar, Niloofar Mireshghallah, Ximing Lu, Maarten Sap, Yejin Choi, and Nouha Dziri. 2024. WildTeaming at Scale: From In-the-Wild Jailbreaks to (Adversarially) Safer Language Models. In *NeurIPS 2024*. arXiv:2406.18510

[18] Mintong Kang and Bo Li. 2025. R^2-Guard: Robust Reasoning Enabled LLM Guardrail via Knowledge-Enhanced Logical Reasoning. In *ICLR 2025*. arXiv:2407.05557

[19] Zi Lin, Zihan Wang, Yongqi Tong, Yangkun Wang, Yuxin Guo, Yujia Wang, and Jingbo Shang. 2023. ToxicChat: Unveiling Hidden Challenges of Toxicity Detection in Real-World User-AI Conversation. In *Findings of EMNLP 2023*. arXiv:2310.17389

[20] Vasudev Majhi, Dhruv Gupta, Advait Singh, Matthew Barker, and Dhruv Kumar. 2025. Do You Really Need a GPU to Guard Your LLM? CPU-Class Classifiers and Multi-Stage Pipelines for Safety Enforcement at Scale. *arXiv preprint* (2025). arXiv:2512.19011 v3 revised 2026; under review.

[21] Todor Markov, Chong Zhang, Sandhini Agarwal, Tyna Eloundou, Teddy Lee, Steven Adler, Angela Jiang, and Lilian Weng. 2023. A Holistic Approach to Undesired Content Detection in the Real World. In *AAAI 2023*. arXiv:2208.03274

[22] Mantas Mazeika, Long Phan, Xuwang Yin, Andy Zou, Zifan Wang, Norman Mu, Elham Sakhaee, Nathaniel Li, Steven Basart, Bo Li, David Forsyth, and Dan Hendrycks. 2024. HarmBench: A Standardized Evaluation Framework for Automated Red Teaming and Robust Refusal. In *ICML 2024*. arXiv:2402.04249

[23] Meta (Llama / PurpleLlama Team). 2024. Prompt Guard (Llama Prompt Guard): mDeBERTa-based prompt-injection / jailbreak classifier. Meta model card (PurpleLlama), https://github.com/meta-llama/PurpleLlama/blob/main/Prompt-Guard/MODEL_CARD.md. model card only; no arXiv paper.

[24] Meta (Llama Team), Grattafiori, et al. 2024. Llama Guard 2 / Llama Guard 3 (described in "The Llama 3 Herd of Models"). *arXiv preprint / Meta model cards* (2024). arXiv:2407.21783 Llama 3 Herd; model cards: https://github.com/meta-llama/PurpleLlama.

[25] Inkit Padhi, Manish Nagireddy, Giandomenico Cornacchia, Subhajit Chaudhury, Tejaswini Pedapati, Pierre Dognin, Keerthiram Murugesan, Erik Miehling, Martín Santillán Cooper, Kieran Fraser, Giulio Zizzo, Muhammad Zaid Hameed, Mark Purcell, Michael Desmond, Qian Pan, Zahra Ashktorab, Inge Vejsbjerg, Elizabeth M. Daly, Michael Hind, Werner Geyer, Ambrish Rawat, Kush R. Varshney, and Prasanna Sattigeri. 2025. Granite Guardian: Comprehensive LLM Safeguarding. In *Proceedings of NAACL 2025 (Industry Track)*. arXiv:2412.07724

[26] Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. 2023. Direct Preference Optimization: Your Language Model is Secretly a Reward Model. In *NeurIPS 2023*. arXiv:2305.18290

[27] Paul Röttger, Hannah Rose Kirk, Bertie Vidgen, Giuseppe Attanasio, Federico Bianchi, and Dirk Hovy. 2024. XSTest: A Test Suite for Identifying Exaggerated Safety Behaviours in Large Language Models. In *NAACL 2024*. arXiv:2308.01263

[28] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. 2017. Proximal Policy Optimization Algorithms. *arXiv preprint* (2017). arXiv:1707.06347

[29] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y.K. Li, Y. Wu, and Daya Guo. 2024. DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models. *arXiv preprint* (2024). arXiv:2402.03300

[30] Wenjun Zeng, Yuchi Liu, Ryan Mullins, Ludovic Peran, Joe Fernandez, Hamza Harkous, Karthik Narasimhan, Drew Proud, Piyush Kumar, Bhaktipriya Radharapu, Olivia Sturman, and Oscar Wahltinez. 2024. ShieldGemma: Generative AI Content Moderation Based on Gemma. *arXiv preprint* (2024). arXiv:2407.21772